

Linux for Computational Chemistry

Lecture 5: Software Environments, Version Control, and Putting It All Together

Emilio Martínez Núñez

Master in Theoretical and Computational Chemistry



conda env

Outline

Package and environment managers

Compiling and PATH

Version control with Git

The full workflow

Package and environment managers

System-level: apt (Debian/Ubuntu)

```
sudo apt update  
sudo apt install build-essential gnuplot
```

Note

On a shared HPC cluster you almost never have `sudo`. System packages are installed by administrators; you manage your *own* software with a user-level tool instead — typically `conda`/`mamba`.

User-level: conda / mamba environments

```
conda create -n comp_chem python=3.11 numpy ase
conda activate comp_chem
conda install -c conda-forge rdkit
conda deactivate
conda env list
```

- An **environment** is an isolated set of packages — no admin rights needed
- mamba is a drop-in, much faster reimplementations of the same commands
- Keep one environment per project to avoid version conflicts between tools

Exercise 5.1 — A project environment

Exercise 5.1

1. Create a conda environment named `md_project` with Python 3.11 and the packages `numpy` and `matplotlib`.
2. Activate it and confirm with `python -version` and `conda list`.
3. Add the package `ase` to the already-created environment.
4. Deactivate it and list all your environments.

Solution

```
conda create -n md_project python=3.11 numpy matplotlib
conda activate md_project
python --version
conda list
conda install ase
conda deactivate
conda env list
```

Compiling and PATH

Compiling software from source (overview)

```
tar -xzf somecode-1.0.tar.gz
cd somecode-1.0
./configure --prefix=$HOME/software/somecode
make
make install
```

- Many quantum chemistry codes still require this classic `configure / make / make install` cycle
- `-prefix` controls *where* it installs, since you have no `/usr` write access
- Some modern packages use `cmake` instead of `configure` — same idea, different tool

PATH and .bashrc

```
echo $PATH # colon-separated list of directories  
export PATH="$HOME/software/somecode/bin:$PATH" # add a directory (this session only)
```

```
# add to the end of ~/.bashrc to make it permanent:  
export PATH="$HOME/software/somecode/bin:$PATH"
```

Tip

.bashrc runs every time you open a new shell — the standard place to permanently customize your environment on a machine you control.

Exercise 5.2 — Customizing your PATH

Exercise 5.2

1. Create a directory `$HOME/bin` and move (or symlink) one of your scripts from Lecture 3, e.g. `check_jobs.sh`, into it.
2. Add `$HOME/bin` to your `PATH` for the current session and confirm you can run `check_jobs.sh` from anywhere without `./`.
3. Add the same line to `~/.bashrc` so it persists in new terminals.

Solution

```
mkdir -p $HOME/bin  
mv check_jobs.sh $HOME/bin/  
export PATH="$HOME/bin:$PATH"  
cd /tmp && check_jobs.sh  
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc
```

Version control with Git

Why version control?

- Your scripts and input files change over a whole master's thesis — Git records every change, who made it, and why
- Undo mistakes: go back to any earlier working version
- Share and collaborate without emailing zip files back and forth
- A **repository** is a directory Git is tracking; commits are labeled snapshots

The core Git commands

```
git init # start tracking this directory
git status # what changed since the last commit?
git add script.sh # stage a file for the next commit
git add . # stage everything changed
git commit -m "Add check_jobs.sh" # record a snapshot
git log # history of commits
git diff # unstaged changes, line by line
```

Exercise 5.3

1. Initialize a Git repository in your `2-butanol_project` directory from Lecture 1.
2. Add and commit `check_jobs.sh` with a meaningful commit message.
3. Edit the script (add a comment line), then use `git diff` to see the change before committing it.
4. Commit the edit, then view the full history with `git log`.

Solution

```
cd 2-butanol_project
git init
git add scripts/check_jobs.sh
git commit -m "Add check_jobs.sh"
# ... edit the file, add a comment ...
git diff
git add scripts/check_jobs.sh
git commit -m "Document check_jobs.sh"
git log
```

Remotes and .gitignore

```
git clone https://github.com/yourgroup/butanol-project.git
git push origin main
git pull origin main
```

```
# .gitignore
*.log
*.chk
slurm-*.out
```

Note

Track your *scripts and inputs* in Git; keep large or regenerable outputs (.log, .chk, SLURM output) out of the repository with .gitignore.

Exercise 5.4

1. In your repository, create a `.gitignore` file that excludes `*.log`, `*.chk`, and `slurm-*.out`.
2. Run `git status` before and after adding it, on a directory that already contains some `.log` files, and confirm the difference.
3. Commit the `.gitignore` file itself.

Solution

```
git status # .log files show as untracked
cat > .gitignore << EOF
*.log
*.chk
slurm-*.out
EOF
git status # .log files no longer listed
git add .gitignore
git commit -m "Add .gitignore for generated output files"
```

The full workflow

Everything together

- | | |
|--------------------------|---|
| 1. Edit script/input | <code>nano</code> , tracked with <code>git</code> |
| 2. Prepare environment | <code>module load</code> / <code>conda activate</code> |
| 3. Submit to the cluster | <code>sbatch run.slurm</code> (possibly an array) |
| 4. Monitor | <code>squeue</code> , <code>sacct</code> |
| 5. Post-process | <code>grep/sed/awk</code> pipeline, or <code>check_jobs.sh</code> |
| 6. Record results | commit updated scripts and summary tables to <code>git</code> |

Note

Every tool from all five lectures fits into exactly one stage of this cycle. This *is* the day-to-day workflow of computational chemistry research on a cluster.

Exercise 5.5

Starting from your `2-butanol_project` repository:

1. Write a SLURM array script to submit 5 conformer optimizations.
2. After they complete, run your `check_jobs.sh` to build `summary.csv`.
3. Sort `summary.csv` by energy and identify the lowest-energy conformer with a one-line `awk` command.
4. Commit the updated scripts and `summary.csv` to Git with a descriptive message, having first added a `.gitignore` for the raw `.log` files.

This single exercise touches every lecture in the course.

- L1: navigating the filesystem, permissions
- L2: redirection, grep/sed/awk, pipelines
- L3: bash scripting — variables, conditionals, loops, functions
- L4: remote access, modules, SLURM job submission and arrays
- L5: conda environments, compiling, PATH, Git, and the full workflow

- *The Linux Command Line* by William Shotts (free online)
- Software Carpentry: <https://software-carpentry.org>
- Your cluster's own documentation for partitions, quotas, and available modules
- `man` and `-help` — still, always, the fastest answer